

ESTRUCTURAS DE DATOS Y PROGRAMACIÓN EN C

Apunte consolidado: Árboles, Listas Enlazadas, Listas Dobles, Listas Circulares, Estructuras Dinámicas, Punteros y Funciones

1. ESTRUCTURAS DE DATOS: CONCEPTOS GENERALES

1.1 Definición

Una estructura de datos es una forma de organizar y almacenar información en la memoria de una computadora, de manera que pueda ser utilizada de forma eficiente. Existen dos grandes clasificaciones: por la forma en que se almacenan en memoria (estáticas o dinámicas) y por la relación entre sus elementos (lineales o no lineales).

1.2 Estructuras estáticas vs. dinámicas

Estructuras de datos estáticas: son aquellas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute (en tiempo de compilación) y no puede modificarse durante la ejecución. Ejemplos: arreglos (arrays) y registros (structs) de tamaño fijo.

Estructuras de datos dinámicas: son aquellas en las que el tamaño puede modificarse durante la ejecución del programa; teóricamente no hay límite a su tamaño, salvo el que impone la memoria disponible en la computadora. Ejemplos: listas enlazadas, árboles, pilas y colas implementadas con punteros.

Se las llama estructuras dinámicas porque pueden cambiar tanto de forma como de tamaño durante la ejecución del programa.

1.3 Estructuras lineales vs. no lineales

Estructuras lineales: son aquellas en las que los elementos ocupan lugares sucesivos y cada uno tiene un único sucesor y un único predecesor (sus elementos están ubicados uno al lado del otro, relacionados linealmente). Ejemplos: arreglos, registros, pilas, colas, listas.

Estructuras no lineales: son aquellas en las que cada elemento puede tener más de un sucesor. Ejemplos: árboles, gráficas (grafos).

1.4 Punteros

Un **puntero** es una variable que contiene la dirección de memoria de otra variable. Es un tipo de dato que "apunta" a otro valor almacenado en memoria; en lugar de contener el dato directamente, contiene la ubicación (dirección) donde ese dato reside.

Condiciones que se deben cumplir para trabajar con punteros en C:

- Un puntero debe declararse indicando el tipo de dato al que apunta, usando el operador *.
Ejemplo: `struct Nodo* siguiente;`
- El operador & se usa para obtener la dirección de memoria de una variable.
- El operador * (de desreferencia) se usa para acceder al valor almacenado en la dirección a la que apunta el puntero.
- Un puntero debe inicializarse (por ejemplo, en NULL) antes de usarse, para evitar comportamientos indefinidos.
- Para reservar memoria dinámica para que un puntero apunte a ella se utiliza malloc (o calloc); para liberar esa memoria se utiliza free.

2. LISTAS ENLAZADAS SIMPLES

2.1 Definición

Una lista simplemente enlazada (también llamada lista lineal o lista enlazada simple) es una estructura de datos lineal y dinámica que utiliza un conjunto de nodos para almacenar datos en secuencia, enlazados mediante un apuntador o referencia.

2.2 Composición de un nodo

Cada nodo de una lista simple tiene 2 secciones:

- **Dato:** puede ser simple (un entero, un carácter) o compuesto (un struct con varios campos).
- **Apuntador o referencia** que enlaza al siguiente nodo en la secuencia lógica.

Condiciones que debe cumplir la estructura: la lista simple tiene un apuntador inicial (comúnmente llamado PRIM o cabeza) que señala al primer nodo, y el último nodo de la lista siempre debe apuntar a NULL (nulo), lo que indica el final del recorrido.

2.3 Definición de la estructura del nodo en C

```
struct Nodo
{
    int dato;
    struct Nodo* siguiente;
};
```

- *int dato*: almacena un valor (puede ser cualquier tipo, no solo entero).
- *struct Nodo* siguiente*: apunta al siguiente nodo de la lista. En el último nodo, este campo debe valer NULL.

2.4 Función crearNodo / nuevo nodo

Para crear un nodo es necesario reservar memoria dinámica con malloc, asignar el valor recibido al campo dato, e inicializar el puntero siguiente en NULL (porque todavía no está conectado a ningún otro nodo). Debe verificarse siempre que malloc no devuelva NULL (lo que indicaría que no hay memoria disponible).

```
struct Nodo* crearNodo(int valor)
{
    struct Nodo* nuevo = (struct Nodo*) malloc(sizeof(struct Nodo));
    nuevo->dato = valor;
    nuevo->siguiente = NULL;
    return nuevo;
}
```

2.5 Operaciones básicas con listas simples

Las operaciones fundamentales que debe soportar una lista enlazada simple son:

- Crear una lista.
- Insertar un elemento en la lista (al principio, al final o en el medio).
- Eliminar un elemento de la lista (al principio, al final o en el medio).
- Mostrar (recorrer) los elementos de la lista.
- Buscar un elemento dentro de la lista.

2.6 Almacenamiento de los datos en una lista simple

Ordenados: se recorren lógicamente los nodos de la lista para ubicar la posición definitiva del nuevo nodo, de manera que se mantenga el orden lógico de los datos (por ejemplo, de menor a mayor).

Desordenados: el nuevo dato simplemente se agrega al final (o al principio) de la lista, sin tener en cuenta ningún criterio de orden.

2.7 Insertar un nodo al principio de la lista

Condición/algorithm: se crea el nuevo nodo p, se le asigna el dato, su puntero "siguiente" debe apuntar a lo que actualmente es el primer nodo (prim), y luego el puntero externo prim debe actualizarse para apuntar al nuevo nodo p. El orden de estos pasos es crítico: si se actualiza prim antes de enlazar el nuevo nodo, se pierde la referencia al resto de la lista.

```
nuevo(p);
si (p = null) entonces escribir("Error");
sino
    p->dato = dato;
    p->siguiente = prim;
    prim = p;
```

2.8 Insertar un nodo al final de la lista

Condición/algorithm: se debe recorrer la lista completa desde el primer nodo hasta encontrar el último (aquel cuyo puntero siguiente es NULL). El nuevo nodo se enlaza desde ese último nodo, y el puntero siguiente del nuevo nodo debe quedar en NULL, ya que pasa a ser el nuevo último elemento.

```
nuevo(p);
si (p = null) entonces escribir("Error");
sino
    t = prim;
    mientras (t <> Null) hacer
        aux = t;
        t = t->proximo;
    fin mientras
    p->dato = dato;
    p->proximo = null;
    aux->proximo = p;
```

2.9 Insertar un nodo en el medio (lista ordenada)

Condición: se recorre la lista comparando el dato a insertar con los valores existentes, hasta encontrar la posición donde el dato del nodo actual ya no es menor que el dato a insertar (o se llega al final). El nuevo nodo se inserta entre el nodo anterior (aux) y el nodo actual (t), respetando el orden.

```
nuevo(p);
si (p = null) entonces escribir("Error");
sino
    t = prim;
    mientras (t <> Null) y (t->valor < dato) hacer
        aux = t;
        t = t->proximo;
    fin mientras
    si (t = null) entonces escribir("No se encontró posición");
    sino
        p->valor = dato;
        p->proximo = t;
        aux->proximo = p;
```

2.10 Eliminar el primer elemento de la lista

Condición: se guarda en un puntero auxiliar t la referencia al primer nodo, se actualiza prim para que apunte al segundo nodo (prim->proximo), y luego se libera la memoria del nodo eliminado con la función disponer/free.

```
t = prim;
prim = t->proximo;
disponer(t);
```

2.11 Eliminar el último elemento de la lista

Condición: hay que recorrer la lista guardando siempre el nodo anterior (aux) hasta llegar al último nodo (aquel cuyo puntero proximo es NULL). Una vez localizado, el puntero proximo del anteúltimo nodo debe pasar a NULL, y se libera la memoria del último nodo.

```
t = prim;
mientras (t->proximo <> Null) hacer
    aux = t;
    t = t->proximo;
fin mientras
aux->proximo = null;
disponer(t);
```

2.12 Eliminar un elemento en el medio de la lista

Condición: se recorre la lista buscando el nodo cuyo dato coincide con el valor a eliminar, manteniendo siempre el puntero al nodo anterior (aux). Si se encuentra, el nodo anterior debe enlazarse directamente con el nodo siguiente al eliminado, evitando así perder la continuidad de la lista, y luego se libera la memoria del nodo eliminado. Si no se encuentra el elemento, debe informarse que no existe en la lista.

```
t = prim;
mientras (t <> Null) y (t->valor <> dato) hacer
    aux = t;
    t = t->proximo;
fin mientras
si (t = null) entonces escribir("No se encontró el elemento");
sino
    aux->proximo = t->proximo;
    disponer(t);
```

2.13 Buscar un elemento en la lista

Condición: se recorre secuencialmente la lista desde el primer nodo, comparando el dato de cada nodo con el valor buscado, hasta encontrarlo o hasta llegar a NULL (fin de la lista, lo que indica que el elemento no existe).

```
t = prim;
mientras (t <> Null) y (t->dato <> dato_buscado) hacer
    t = t->proximo;
fin mientras
si (t = null) entonces escribir("Elemento no encontrado");
sino escribir("Elemento encontrado");
```

2.14 Función agregarAlFinal con doble puntero

Para que una función modifique el puntero externo (cabeza) que vive en main, no alcanza con pasarle un struct Nodo*: hay que pasarle la dirección de ese puntero, es decir un puntero a puntero (struct

Nodo**). La declaración `struct Nodo** cabeza` define `cabeza` como un puntero a un puntero de tipo `Nodo`; `cabeza` almacena la dirección de memoria de otro puntero, que a su vez apunta a una estructura `Nodo`. Esto es necesario en C porque el paso de parámetros es por valor: sin el doble puntero, los cambios sobre `cabeza` dentro de la función no se reflejarían fuera de ella.

Condiciones de la función: si la lista está vacía (`*cabeza == NULL`), el nuevo nodo pasa a ser el primero. Si no está vacía, debe recorrerse hasta el último nodo y enlazarlo al nuevo nodo.

3. LISTAS CIRCULARES

Una lista circular es una estructura de datos lineal donde el último elemento apunta al primer elemento en lugar de terminar en NULL. Esto crea un ciclo o "anillo" continuo, en el que recorriendo la lista siempre se vuelve al punto de partida.

Existen principalmente dos tipos:

- **Listas circulares simples:** permiten un recorrido unidireccional (solo hacia adelante); el último nodo apunta al primero.
- **Listas circulares dobles:** el último nodo apunta al primero y, a su vez, el primero apunta al último, permitiendo moverse en ambas direcciones de manera continua.

Condición clave a tener en cuenta al programar listas circulares: la condición de corte de los recorridos no puede ser "hasta llegar a NULL" (porque nunca se llega a NULL), sino "hasta volver al nodo de partida". Si no se controla esta condición correctamente, se genera un bucle infinito.

4. LISTAS DOBLEMENTE ENCADENADAS

4.1 Definición

Una lista doblemente encadenada es una estructura de datos dinámica que se compone de un conjunto de nodos en secuencia, enlazados mediante dos apuntadores: uno hacia adelante (al siguiente nodo) y otro hacia atrás (al nodo anterior).

4.2 Composición de un nodo doble

Cada nodo de una lista doble tiene 3 secciones:

- **Dato** (puede ser simple o compuesto).
- **Apuntador o referencia que enlaza al siguiente nodo** en secuencia lógica.
- **Apuntador que enlaza al nodo anterior** en secuencia lógica.

Condiciones estructurales: la lista doble tiene un apuntador inicial (PRIM) y un apuntador final (ULT). El puntero "anterior" del primer nodo y el puntero "siguiente" del último nodo deben apuntar siempre a NULL (nulo).

4.3 Definición de la estructura del nodo en C

```
struct Nodo {  
    int dato;  
    struct Nodo* siguiente;  
    struct Nodo* anterior;  
};
```

4.4 Operaciones básicas con listas doblemente enlazadas

- Añadir o insertar elementos (al inicio, al final o en el medio).
- Buscar o localizar elementos.
- Borrar elementos.
- Moverse a través de la lista en ambos sentidos: hacia el siguiente y hacia el anterior.

4.5 Insertar un nodo al inicio de la lista

Condición/algorithm: el nuevo nodo p debe tener su puntero anterior en NULL (porque pasará a ser el primero), y su puntero posterior debe apuntar al nodo que actualmente es el primero (prim). Además, hay que actualizar el puntero anterior del antiguo primer nodo para que apunte a p, y finalmente actualizar prim para que apunte a p. Es fundamental actualizar los cuatro enlaces en el orden correcto para no perder ninguna referencia.

```
nuevo(p);  
p->dato = dato;  
p->anterior = Null;  
p->posterior = prim;  
prim->anterior = p;  
prim = p;
```

4.6 Insertar un nodo al final de la lista

Condición/algorithm: el nuevo nodo p debe tener su puntero posterior en NULL (porque pasará a ser el último), y su puntero anterior debe apuntar al nodo que actualmente es el último (ult). Hay que actualizar el puntero posterior del antiguo último nodo para que apunte a p, y finalmente actualizar ult

para que apunte a p.

```
nuevo(p);
p->dato = dato;
p->posterior = Null;
p->anterior = ult;
ult->posterior = p;
ult = p;
```

4.7 Insertar un nodo en el medio de la lista

Condición/algorithm: se recorre la lista desde prim contando posiciones (con un contador i) hasta llegar a la posición deseada (pos) o hasta agotar la lista. Si se encuentra la posición, el nuevo nodo p debe enlazarse: su anterior apunta al nodo previo (anterior), su posterior apunta al nodo actual (t); el nodo anterior debe actualizar su puntero posterior hacia p, y el nodo actual debe actualizar su puntero anterior hacia p. Si t llega a NULL sin encontrar la posición, debe informarse el error.

```
t = prim;
i = 1;
mientras (t <> null) y (i < pos) hacer
    anterior = t;
    t = t->posterior;
    i++;
fin mientras

si (t = null) entonces escribir("No se encontró posición");
sino
    nuevo(p);
    si (p = null) entonces escribir("No hay espacio");
    sino
        p->dato = dato;
        p->anterior = anterior;
        p->posterior = t;
        t->anterior = p;
        anterior->posterior = p;
```

4.8 Buscar un elemento en la lista doble

Condición: por eficiencia, primero se compara el dato buscado contra el primer nodo (prim) y contra el último (ult), ya que el acceso a ambos extremos es directo. Si no coincide con ninguno de los dos, se recorre la lista secuencialmente desde prim comparando cada nodo, hasta encontrarlo o hasta llegar a NULL.

```
ingresar(dato_bus);
si (prim->dato = dato_bus) entonces escribir("Elemento encontrado");
sino
    si (ult->dato = dato_bus) entonces escribir("Elemento encontrado");
    sino
        t = prim;
        mientras (t <> null) y (t->dato <> dato_bus) hacer
            t = t->posterior;
        fin mientras
        si (t = null) escribir("Elemento no encontrado");
        sino escribir("Elemento encontrado");
```

4.9 Eliminar el primer elemento de la lista doble

Condición: se guarda en p el segundo nodo (prim->posterior), se libera el nodo prim original, se actualiza el puntero anterior del nuevo primer nodo (p) a NULL, y finalmente prim pasa a apuntar a p.

```
p = prim->posterior;
liberar(prim);
p->anterior = Null;
prim = p;
```

4.10 Eliminar el último elemento de la lista doble

Condición: se guarda en p el penúltimo nodo (ult->anterior), se libera el nodo ult original, se actualiza el puntero posterior del nuevo último nodo (p) a NULL, y finalmente ult pasa a apuntar a p.

```
p = ult->anterior;
liberar(ult);
p->posterior = Null;
ult = p;
```

4.11 Eliminar un elemento en el medio de la lista doble

Condición/algoritmo: se recorre la lista contando posiciones, guardando el nodo anterior, hasta llegar a la posición deseada o agotar la lista. Si se encuentra (t distinto de NULL), el nodo anterior debe enlazar directamente su puntero posterior con el nodo siguiente al eliminado (t->posterior); y ese nodo siguiente (aux) debe actualizar su puntero anterior para que apunte al nodo "anterior" (saltando el nodo eliminado en ambas direcciones). Por último se libera la memoria del nodo eliminado.

```
t = prim;
i = 1;
mientras (t <> null) y (i < pos) hacer
    anterior = t;
    t = t->posterior;
    i++;
fin mientras

si (t = null) entonces escribir("No se encontró posición");
sino
    anterior->posterior = t->posterior;
    aux = t->posterior;
    aux->anterior = anterior;
    liberar(t);
```

5. ÁRBOLES

5.1 Definición

Un árbol se puede definir como una estructura jerárquica y en forma no lineal, aplicada sobre una colección de elementos u objetos llamados nodos (Cairó & Guardati, 2006).

Los árboles son considerados estructuras de datos no lineales y dinámicas muy importantes en el área de computación. Son muy utilizados como un método eficiente para búsquedas grandes y complejas; casi todos los sistemas operativos almacenan sus archivos en árboles o estructuras similares a árboles.

Se les llama estructuras dinámicas porque pueden cambiar de forma y de tamaño durante la ejecución del programa, y estructuras no lineales porque cada elemento del árbol puede tener más de un sucesor.

5.2 Terminología: relación con otros nodos

- **Nodo:** cada elemento que contiene el árbol.
- **Nodo padre:** todo nodo que tiene al menos un hijo.
- **Nodo hijo:** todo nodo que tiene un padre.
- **Nodo hermano:** nodos que comparten un mismo padre en común dentro de la estructura.

5.3 Terminología: posición dentro del árbol

- **Nodo raíz:** el primer nodo de un árbol. Solo un nodo del árbol puede ser la raíz.
- **Nodo hoja:** todo nodo que no tiene hijos; siempre se encuentran en los extremos de la estructura.
- **Nodo interior o rama:** todo nodo que no es la raíz y que además tiene al menos un hijo.

5.4 Terminología: tamaño del árbol

Nivel: el nivel de un nodo es su distancia a la raíz. Condiciones: un árbol vacío tiene 0 niveles; el nivel de la raíz es 1; el nivel de cada nodo se calcula contando cuántos nodos existen sobre él hasta llegar a la raíz, más 1 (o, de forma inversa, contando cuántos nodos existen desde la raíz hasta el nodo buscado, más 1).

Altura: es el número máximo de niveles que tiene un árbol.

Peso: es el número total de nodos que tiene un árbol (la suma de todos los nodos de todos los niveles).

5.5 Árboles binarios

Un árbol binario es aquel en el que cada nodo puede tener como máximo 2 hijos; dicho de otra manera, es un árbol de grado dos.

Estructura del nodo en C:

```
struct Nodo {
    int dato;
    struct Nodo* izquierdo;
    struct Nodo* derecho;
};
```

Campos: dato (de tipo entero u otro tipo) corresponde al valor que contiene el nodo; izquierdoPtr y derechoPtr son punteros a los subárboles izquierdo y derecho respectivamente.

5.6 Árboles binarios distintos, similares y equivalentes

- **Árboles binarios distintos:** dos árboles binarios son distintos cuando sus estructuras son diferentes.
- **Árboles binarios similares:** dos árboles binarios son similares cuando sus estructuras son idénticas, pero la información que contienen sus nodos difiere entre sí.
- **Árboles binarios equivalentes:** son aquellos que son similares y además sus nodos contienen la misma información.

5.7 Árboles binarios completos

Un árbol binario completo de profundidad n es un árbol en el que, para cada nivel del 0 al nivel $n-1$, hay un conjunto lleno de nodos, y todos los nodos hoja del nivel n ocupan las posiciones más a la izquierda del árbol.

La profundidad de un árbol binario es la longitud del camino más largo desde la raíz hasta el nodo hoja más alejado, y se calcula contando el número de nodos en esa ruta. Si el árbol está vacío, la profundidad es 0.

5.8 Árboles binarios equilibrados (balanceados)

Cuando un árbol binario de búsqueda crece descontroladamente hacia un extremo, su rendimiento puede disminuir considerablemente (las búsquedas dejan de ser eficientes y se comportan como una lista enlazada). Para mantener la eficiencia de operación surgen los árboles equilibrados o balanceados, que pueden realizar acomodos o balanceos después de inserciones o eliminaciones de elementos, de modo que la diferencia de altura entre subárboles se mantenga controlada.

5.9 Árbol binario de búsqueda (ABB)

Un árbol binario de búsqueda es aquel cuyos nodos están ordenados de tal manera que los datos del subárbol izquierdo de cualquier nodo son menores que el dato de ese nodo, y los del subárbol derecho son mayores. Esta condición debe cumplirse para todos y cada uno de los nodos del árbol, no solo para la raíz.

Gracias a esta propiedad, sobre un árbol binario de búsqueda se pueden realizar búsquedas de manera similar a la búsqueda binaria utilizada en arreglos, descartando en cada paso la mitad de las posibilidades restantes.

Para crear un árbol binario de búsqueda a partir de un listado de datos: se asume que el primer dato es la raíz del árbol; los datos siguientes se ubican en el árbol de la siguiente manera: los menores que el nodo actual se insertan como hijos por el lado izquierdo, y los mayores se insertan como hijos por el lado derecho, repitiendo la comparación recursivamente hasta encontrar un lugar vacío.

5.10 Recorridos de árboles binarios

Recorrido INORDEN: primero se recorre el subárbol izquierdo, luego se visita la raíz, y por último se recorre el subárbol derecho. En síntesis: hijo izquierdo — raíz — hijo derecho. En un árbol binario de búsqueda, este recorrido devuelve los elementos ordenados de menor a mayor.

Recorrido PREORDEN: primero se visita la raíz, luego se recorre el subárbol izquierdo, y por último el subárbol derecho. En síntesis: raíz — hijo izquierdo — hijo derecho.

Recorrido POSORDEN: primero se recorre el subárbol izquierdo, luego el subárbol derecho, y por último se visita la raíz. En síntesis: hijo izquierdo — hijo derecho — raíz.

Condición común a los tres recorridos: son procedimientos recursivos. El caso base de la recursión es siempre que el nodo (o subárbol) sea NULL, momento en el cual la función debe simplemente retornar sin hacer nada.

5.11 Funciones típicas para implementar un árbol binario en C

Crear un nodo: se reserva memoria con malloc, se asigna el dato recibido, y se inicializan los punteros izquierdo y derecho en NULL.

```
struct Nodo* crearNodo(int valor)
{
    struct Nodo* nuevo = (struct Nodo*) malloc(sizeof(struct Nodo));
    nuevo->dato = valor;
    nuevo->izquierdo = NULL;
    nuevo->derecho = NULL;
    return nuevo;
}
```

Recorrer (imprimir) un árbol: función recursiva que respeta el orden elegido (inorden, preorden o posorden). El caso base es que el nodo sea NULL.

```
void imprimirInorden(struct Nodo* raiz)
{
    if (raiz == NULL) return;
    imprimirInorden(raiz->izquierdo);
    printf("%d ", raiz->dato);
    imprimirInorden(raiz->derecho);
}
```

Insertar en un árbol binario de búsqueda: función recursiva que compara el valor a insertar con el nodo actual; si es menor avanza por la izquierda, si es mayor avanza por la derecha, hasta encontrar una posición NULL donde insertar el nuevo nodo.

```
struct Nodo* insertar(struct Nodo* raiz, int valor)
{
    if (raiz == NULL) {
        return crearNodo(valor);
    }
    if (valor < raiz->dato) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    } else if (valor > raiz->dato) {
        raiz->derecho = insertar(raiz->derecho, valor);
    }
    return raiz;
}
```

Liberar (destruir) un árbol: debe liberarse cada nodo individualmente mediante una función recursiva en posorden (primero los hijos, después el propio nodo), para no perder la referencia a los nodos descendientes antes de liberarlos.

```
void liberarArbol(struct Nodo* raiz)
{
    if (raiz == NULL) return;
    liberarArbol(raiz->izquierdo);
    liberarArbol(raiz->derecho);
    free(raiz);
}
```

6. FUNCIONES EN LENGUAJE C

6.1 Concepto de función

Una función es un subprograma que realiza una tarea determinada y bien acotada, a la cual se le pasan datos y que nos devuelve datos. La función se ejecuta cuando se la llama ("llamada a la función").

6.2 Ventajas de modularizar con funciones

- El programa es más simple de comprender, ya que cada módulo se dedica a realizar una tarea en particular.
- La depuración (debug) queda acotada a cada módulo.
- Las modificaciones al programa se reducen a modificar determinados módulos.
- Cuando cada módulo está bien probado, se lo puede usar las veces que sea necesario sin volver a revisarlo.
- Se obtiene una independencia del código en cada módulo.

6.3 Modularidad

Para resolver un problema complejo de desarrollo de software, conviene separarlo en partes más pequeñas, que se puedan diseñar, desarrollar, probar y modificar de manera sencilla, y lo más independientemente posible del resto de la aplicación. Esas partes se denominan, de manera genérica, módulos. En programación estructurada se modulariza la solución, es decir, el "cómo" del desarrollo.

6.4 Cohesión

La cohesión se refiere a que cada módulo del sistema se refiera a un único proceso o entidad. A mayor cohesión, mejor: el módulo será más sencillo de diseñar, programar, probar y mantener. Se logra alta cohesión cuando cada función o procedimiento realiza una única tarea, trabajando sobre una sola estructura de datos.

Condición/test de cohesión: un módulo es cohesivo si puede describirse con una oración simple, con un solo verbo activo. Si en la descripción del módulo aparece más de un verbo activo, debería analizarse su partición en más de un módulo, y repetir el test.

6.5 Acoplamiento

El acoplamiento mide el grado de relacionamiento de un módulo con los demás. A menor acoplamiento, mejor: el módulo será más sencillo de diseñar, programar, probar y mantener. Se logra bajo acoplamiento reduciendo las interacciones entre procedimientos y funciones, reduciendo la cantidad y complejidad de los parámetros, y disminuyendo al mínimo los parámetros por referencia y los efectos colaterales.

6.6 Características de las funciones en C

- Una de las funciones del programa tiene que llamarse main; la ejecución del programa siempre comienza por las instrucciones contenidas en main.
- Se pueden subordinar funciones adicionales a main, y posiblemente unas a otras.

- Si un programa contiene varias funciones, sus definiciones pueden aparecer en cualquier orden, pero deben ser independientes entre sí: la definición de una función no puede estar incluida dentro de otra.
- Cuando se llama a una función, se ejecutan las instrucciones de las que consta. Se puede acceder a una misma función desde varios lugares distintos del programa.
- Una vez completada la ejecución de una función, se devuelve el control al punto desde el que se la llamó.

6.7 Argumentos, parámetros y la instrucción return

La información se le pasa a la función mediante identificadores especiales llamados argumentos (también denominados parámetros), y se devuelve por medio de la instrucción return. Algunas funciones aceptan información pero no devuelven nada (por ejemplo, la función de biblioteca printf, que es de tipo void).

Si la función es de un tipo de dato distinto de void, en el cuerpo de la función debe escribirse al menos una sentencia return. A continuación de return debe ir el valor que devuelve la función, siempre del mismo tipo (o compatible) al tipo declarado de la función. La forma general es: `return [expresión];`. La instrucción return fuerza la salida inmediata del cuerpo de la función y se vuelve a la siguiente sentencia después de la llamada. Si el tipo de dato de la expresión del return no coincide con el tipo de la función, ese valor se convierte automáticamente al tipo de dato de la función.

Según su declaración, una función devuelve solamente un valor. Si fuera necesario devolver más de un valor, debe realizarse por medio de los parámetros, que dejarían de ser sólo de entrada para convertirse en parámetros de entrada y salida (pasaje por referencia).

6.8 Funciones de tipo void / Procedimientos

En el lenguaje C no se habla habitualmente de "procedimientos", sino solo de funciones, pero existen funciones de ambos tipos. Un procedimiento sería, por ejemplo, la función printf, que no se invoca para calcular un valor nuevo, sino para realizar una tarea (mostrar algo) sobre las variables.

En C se usa una función de tipo void para representar un procedimiento. Una función de tipo void no necesariamente tendrá la sentencia return. Si una función de tipo void hace uso de la sentencia return, en ningún caso debe seguir a esa palabra ningún valor: si así fuera, el compilador detectará un error y no compilará el programa.

6.9 Funciones predefinidas vs. funciones definidas por el usuario

Funciones predefinidas: están definidas en el lenguaje de programación (en sus bibliotecas estándar) y pueden utilizarse directamente, sin tener que programarlas. Ejemplo: pow, strlen.

Funciones definidas por el usuario: se utilizan cuando las funciones predefinidas no permiten realizar el tipo de cálculo deseado, y es el propio programador quien debe implementar, mediante estructuras de control adecuadas, la tarea a realizar.

6.10 Variables locales y variables globales

Todas las variables que se encuentran definidas dentro de las llaves de una función (recordar que main también es una función) tienen validez únicamente dentro de dicha función, y se llaman variables locales.

Si una variable puede ser usada desde cualquier función y durante el transcurso de todo el programa, esa es una variable global. Las variables globales se definen fuera de cualquier función, normalmente debajo de los #include que se colocan al comienzo del programa.

6.11 Pasaje de parámetros por valor

Se llama pasaje por valor cuando a la función se le pasa, como parámetro actual, el valor de la variable. Esto implica que la función recibe una copia del dato: cualquier modificación que la función realice sobre ese parámetro no afecta a la variable original en la función que la invocó.

6.12 Pasaje de parámetros por referencia

Se llama pasaje por referencia cuando a la función se le pasa, como parámetro actual, la dirección de memoria de una variable (obtenida con el operador &). La variable que recibe ese parámetro dentro de la función debe ser, necesariamente, un puntero, ya que lo que se está pasando es una dirección de memoria. Gracias a esto, la función puede modificar directamente el valor de la variable original (accediendo mediante el operador * de desreferencia), y esos cambios sí se reflejan fuera de la función.

6.13 Declaración (prototipo) de una función

Forma general de la declaración de una función:

```
tipo_devuelto nombre_de_funcion(tipo variable_1, tipo variable_2, ..., tipo
variable_N)
{
    <acciones que realiza la función>
}
```

tipo_devuelto: es el tipo de dato que devuelve la función luego de terminada su ejecución (puede ser void si no devuelve nada).

nombre_de_función: es el nombre con el que se identifica y se invoca a la función.

tipo variable_1, ...: son el tipo y el nombre de cada una de las variables (parámetros formales) que se le pasan a la función.

6.14 Reglas de la sentencia de llamada (invocación)

- La sentencia de llamada está formada por el nombre de la función y sus argumentos (los valores que se le pasan), que deben ir entre paréntesis y en el mismo orden que la secuencia de parámetros del prototipo.
- Si la función no recibe parámetros, debe colocarse igualmente, después de su nombre, los paréntesis de apertura y cierre sin ninguna información entre ellos; si no se colocan los paréntesis, se produce un error de compilación.
- El paso de parámetros en la llamada exige una asignación posicional: el valor del primer argumento de la llamada se asigna al primer parámetro formal, el segundo argumento al segundo parámetro formal, y así sucesivamente.
- Debe asegurarse que el tipo de dato de los parámetros formales sea compatible, en cada caso, con el tipo de dato usado en la lista de argumentos de la llamada. El compilador de C no dará error si se fuerzan cambios de tipo de dato incompatibles, pero el resultado será inesperado.
- No es necesario (ni es lo habitual) que los identificadores de los argumentos que se pasan a la función al ser llamada coincidan con los identificadores de los parámetros formales.
- Las llamadas a las funciones pueden realizarse en el orden que sea necesario, y tantas veces como se quiera, independientemente del orden en que hayan sido declaradas o definidas. Una función puede incluso llamarse a sí misma (recursividad).

6.15 Ejemplo completo: función con pasaje por valor

```
#include <stdio.h>

int suma(int a, int b);

int main()
{
    int x, y, z;
    printf("ingrese numero a sumar: ");
    scanf("%d", &x);
    printf("ingrese numero a sumar: ");
    scanf("%d", &y);

    z = suma(x, y);
    printf("La suma es %d", z);
    return 0;
}

int suma(int a, int b)
{
    int total;
    total = a + b;
    return total;
}
```

6.16 Ejemplo completo: función con pasaje por referencia

```
#include <stdio.h>

int suma(int *a, int *b);

int main()
{
    int x, y, z;
    printf("ingrese numero a sumar: ");
    scanf("%d", &x);
    printf("ingrese numero a sumar: ");
    scanf("%d", &y);

    z = suma(&x, &y);
    printf("La suma es %d", z);
    return 0;
}

int suma(int *a, int *b)
{
    int total;
    total = *a + *b;
    return total;
}
```

6.17 Funciones que trabajan con arreglos

Cuando se le pasa un arreglo a una función, en realidad se está pasando un puntero al primer elemento del arreglo; por eso, cuando no se especifica el tamaño del arreglo en su declaración (por ejemplo `int a[]`), el compilador no reserva espacio nuevo, sino que trabaja sobre el arreglo original. Por convención, en estos casos suele pasarse también un segundo parámetro con la cantidad de

elementos del arreglo, ya que la función no tiene forma de saber por sí sola cuántos elementos contiene.

```
int sumoarreglo(int a[], int n)
{
    int suma = 0;
    for (int i = 0; i < n; i++)
        suma += a[i];
    return suma;
}
```

6.18 Recursividad

Una función recursiva es aquella que se llama a sí misma, directa o indirectamente, durante su propia ejecución. Para que una función recursiva funcione correctamente y no entre en un bucle infinito (lo que provocaría desborde de la pila o stack overflow), debe cumplir dos condiciones:

- **Caso base:** debe existir al menos una condición que detenga la recursión sin volver a llamarse a sí misma (por ejemplo, cuando el parámetro llega a 0, o cuando se alcanza un nodo NULL en una lista o árbol).
- **Caso recursivo:** en cada llamada recursiva, el problema debe acercarse progresivamente al caso base (por ejemplo, decrementando un contador, o avanzando al siguiente nodo de una estructura).

Convención de estilo recomendada: dentro del cuerpo de la función, primero debe verificarse el caso base (y retornar si corresponde), y solamente después realizar la acción correspondiente junto con la llamada recursiva. Esto evita procesar de más antes de cortar la recursión.

```
void contador(int numero)
{
    if (numero < 0)
        return;
    printf(" - %d", numero);
    contador(numero - 1);
}
```

7. BUENAS PRÁCTICAS AL PROGRAMAR ESTRUCTURAS DINÁMICAS EN C

Estas son condiciones que, en la práctica, deben cumplirse al implementar listas y árboles en lenguaje C, además de la lógica propia de cada estructura:

- **Verificar el resultado de malloc:** si malloc no puede reservar memoria, devuelve NULL. Antes de usar el puntero devuelto (por ejemplo, antes de hacer strcpy o asignar campos), debe comprobarse que no sea NULL, y manejar el error correspondiente.
- **Liberar toda la memoria reservada:** por cada malloc debe existir, en algún momento del programa, un free correspondiente, para evitar fugas de memoria (memory leaks). En listas, se libera nodo por nodo recorriendo la estructura; en árboles, se libera recursivamente (típicamente en posorden).
- **No usar punteros después de liberarlos (dangling pointers):** luego de hacer free(puntero), no debe volver a accederse al contenido de esa dirección de memoria.
- **Usar funciones seguras para cadenas:** preferir strncpy sobre strcpy, especificando siempre el tamaño máximo del buffer destino, para evitar desbordes de buffer cuando la cadena de origen es más larga que el espacio reservado.
- **Inicializar siempre los punteros:** todo puntero de un nodo nuevo (siguiente, anterior, izquierdo, derecho) debe inicializarse explícitamente, normalmente en NULL, para no dejar referencias indefinidas (basura) que luego provoquen errores difíciles de detectar.
- **Mantener actualizados los punteros externos:** en listas y árboles, los punteros que viven fuera de la estructura (como PRIM, ULT, o la raíz) deben actualizarse en cada operación de inserción o eliminación que afecte al primer o último elemento, o a la raíz.
- **Respetar el orden de las asignaciones de punteros:** al insertar o eliminar nodos, el orden en que se reasignan los punteros es crítico; si se sobrescribe una referencia antes de usarla para enlazar el resto de la estructura, se pierde el acceso a esa parte de la lista o árbol de forma irrecuperable.